NEURAL NETWORKS

LAB 3 | TASK 1

Penguins Binary Classification

Perceptron and Adaline with a Graphical User Interface

DEPARTMENT	**CS**
TEAM	**T09**
TASK	**1**
COURSE MATERIAL	**Neural Networks Lab 3 (2025-2026)**

PROJECT DOCUMENTATION

Prepared from the Lab 3 task specification and the modular Python implementation.

Project Overview

This document describes the implementation of a two-class Penguins classifier for Neural Networks Lab 3, Task 1. The application implements a single-layer Perceptron and ADALINE (Adaptive Linear Neuron), offers a Tkinter graphical interface, plots the learned decision boundary, and evaluates predictions on a held-out test set.

Implementation summary

The program keeps the GUI separate from data loading, preprocessing, model training, evaluation, visualization, and report generation. It preserves all 150 data rows, uses 30 training and 20 testing samples per selected class, and calculates the confusion matrix without scikit-learn metrics.

Team Information

Team member	Student ID
Tageldin Sirrelkhatim Abdelwahid Othman	2023170793
Khalid Yahia Abdalla	2022170563
Maram Salaheldin Elsayd Ali	2023170567
Mohamed Fahim Ragab Fahim	2023170528
Ola Abdlaraheem Abdulaziz Ahmed	2023170780

Project Objectives

- Implement two linear binary classifiers and allow the user to choose the algorithm and training settings.
- Use two selectable features and two selectable Penguin species for each binary experiment.
- Train on a non-repeated, balanced subset and evaluate on the remaining unseen samples.
- Visualize the decision boundary together with training and test observations.
- Provide a manually computed confusion matrix, overall accuracy, single-sample prediction, and an analysis report.

Document Scope

The sections below map the source assignment requirements to the code, explain the implementation decisions, document the reproducible experiment settings, and discuss the five representative plots generated for each algorithm. Reported accuracies belong to the fixed split and settings described in this document.

1. Lab Requirements and Implementation Traceability

The following checklist maps the requirements stated in the uploaded Lab 3 slides to the implemented project files and observable application behavior.

Requirement from Lab 3	How it is addressed	Evidence
Perceptron classifier	Implemented with sample-by-sample classification-error updates.	models.py
ADALINE using MSE	Linear output, stochastic gradient descent and per-epoch MSE check; training may stop at the requested threshold.	models.py
Choose two features	GUI lets the user select two distinct features from the five dataset features.	gui.py, data_loader.py
Choose two of three species	GUI supports Adelie, Chinstrap and Gentoo; two different species are selected for each run.	gui.py, data_loader.py
User training controls	Learning rate, maximum epochs, MSE threshold, Bias checkbox, and algorithm selector are available.	gui.py
Small random initialization	Weights and optional bias are initialized from a small normal distribution; a fixed random seed supports repeatability.	models.py
Single sample classification	A dedicated GUI tab accepts values for the selected feature pair and displays the predicted species and decision score.	gui.py, experiment.py
30 train / 20 test per class	Balanced split: 30 samples per class for training and the remaining 20 per class for testing; no train/test overlap.	data_loader.py
Decision boundary and scatter plot	Plot shows a linear boundary, training samples, and held-out test samples for both classes.	visualization.py
Confusion matrix and overall accuracy	Both are computed with explicit loops; no scikit-learn metrics or built-in confusion-matrix utility is used.	evaluation.py
Preserve every dataset row	All 150 rows are retained on loading; missing numeric measurements are imputed using training-set medians.	data_loader.py, preprocessing.py
Preprocess OriginLocation	Location categories are assigned deterministic numeric codes to keep the selected input two-dimensional.	data_loader.py
At least 5 experiments per algorithm	Automated report selects five representative feature pairs per algorithm and writes results for all 30 feature/class combinations per algorithm.	report_generator.py
Highest-accuracy features	Report identifies highest test accuracy and all feature/class combinations tied at that maximum.	report_generator.py
Separate UI and logic	The Tkinter GUI calls the experiment layer; data, preprocessing, models, metrics, plotting and report generation are separate modules.	Project architecture
Readable, maintainable deliverable	Modules have focused responsibilities and the project includes README, requirements and core unit tests.	Project files

Important constraint

The report generator includes generated figures and CSV outputs in the output folder. For a submission, regenerate them locally from the current code and retain your own screenshots of the GUI if required by the instructor.

2. Dataset and Preprocessing

The project uses the Penguins dataset supplied with the assignment. It contains three species and five selectable input features. The binary classifier trains on exactly two selected species at a time.

Dataset property	Project setting
Source file	data/penguins.csv
Rows retained	150 total rows; no rows are dropped during loading.
Species	Adelie, Chinstrap, Gentoo (50 records per species in the task dataset).
Input features	CulmenLength, CulmenDepth, FlipperLength, OriginLocation, BodyMass.
Target column	Species
Training subset	30 non-repeated samples from each selected species (60 total).
Testing subset	Remaining 20 samples from each selected species (40 total).
Split seed	42, for repeatability under the same dataset and code.

Preprocessing sequence

1. Load and validate the CSV columns while preserving the original number of rows.
2. Map OriginLocation category names to deterministic numeric codes so the feature remains one selected input and the plot remains two-dimensional.
3. Create the balanced train/test split separately within each selected class; the two subsets do not overlap.
4. Fit median imputation and standardization using training data only. Apply the same fitted medians, means, and scales to test samples and the plotting grid.
5. Encode Class A as -1 and Class B as +1 for learning and prediction.

Missing values and data leakage

The CSV includes missing numeric measurements. Rows are not removed. The Standardizer computes medians from the training subset and uses those medians to fill missing values before standardizing the selected features. Means and standard deviations are also learned from the training subset only. This avoids using held-out test measurements to fit preprocessing parameters.

Categorical encoding caveat

OriginLocation is nominal, so the assigned numeric codes are category identifiers rather than meaningful ordered quantities. This scalar encoding is used as a practical compromise to satisfy the assignment's two-feature input and two-dimensional decision-boundary plot requirement. Interpret results involving this feature with that limitation in mind.

3. Algorithms and Learning Rules

3.1 Single-Layer Perceptron

The Perceptron computes the net input from the two standardized features and the learned weights. Its prediction is a hard binary decision: output +1 when the score is at least zero, otherwise output -1.

Decision rule

net = w1*x1 + w2*x2 + b
ŷ = +1 if net >= 0; otherwise ŷ = -1

For each training sample, the model compares the predicted class with the target. When the sample is misclassified, it updates the weights in proportion to the learning rate, the target-prediction difference, and the feature vector. If Bias is enabled, the same error-based update is applied to the bias. Training stops early when an epoch has no misclassified samples or when the maximum epoch count is reached.

3.2 ADALINE (Adaptive Linear Neuron)

ADALINE uses a linear output during learning instead of applying a hard threshold to the net input. It measures the continuous difference between the target and current linear output and uses stochastic gradient descent (one training sample at a time) to reduce mean squared error.

Cost and update

error = target - linear_output
MSE = mean((target - linear_output)^2)
Weights and optional bias are updated after each sample using the learning rate and current error.

After every epoch, the implementation records training MSE. Training ends when the MSE is at or below the configured threshold, or when the maximum number of epochs is reached. The final classifier converts the linear score to -1 or +1 for class prediction.

3.3 Common properties and differences

Aspect	Perceptron	ADALINE
Learning signal	Hard predicted class and classification error.	Continuous linear-output error.
Optimization target	No explicit differentiable cost is minimized by the Perceptron rule.	Minimizes mean squared error (MSE).
Update style	Sample-by-sample, when a sample is misclassified.	Sample-by-sample stochastic gradient descent.
Stopping logic	No mistakes in an epoch or maximum epochs.	MSE threshold reached or maximum epochs.
Final decision	Linear score thresholded at zero.	Linear score thresholded at zero.
Boundary geometry	Straight line in the selected two-feature plane.	Straight line in the selected two-feature plane.

Both algorithms are linear binary classifiers. Feature overlap in the selected two-dimensional view can therefore limit accuracy even when the training code is working correctly.

4. Software Architecture and File Responsibilities

The implementation is modular. The GUI handles user interaction and presentation; the training and analysis logic is kept in independent modules so it can be tested and reused.

File	Responsibility
main.py	Application entry point; launches the GUI.
gui.py	Tkinter widgets, input validation, button actions, result display, single-sample input, and report-launch controls.
data_loader.py	CSV loading and validation, feature/class definitions, OriginLocation encoding, and balanced train/test splitting.
preprocessing.py	Training-only median imputation, feature standardization, and inverse transformation for plotting.
models.py	Shared linear classifier foundation, Perceptron, ADALINE, initialization, fit, and predict methods.
evaluation.py	Manual 2x2 confusion matrix, overall accuracy, and class recalls.
experiment.py	Coordinates splitting, preprocessing, training, test prediction, metrics, and single-sample classification.
visualization.py	Decision-boundary grids, data scatter plots, and training/test marker distinctions.
report_generator.py	Runs feature/class combinations, selects five representative examples per algorithm, writes the CSV and analysis reports, and saves the figures.
generate_report.py	Command-line entry point for generating the full report without opening the GUI.
tests/test_core.py	Core unit tests for data integrity, split sizes, metrics, model runs, and single-sample prediction.
data/penguins.csv	Dataset used by the application.

Data flow

User selections in gui.py -> data_loader.py creates a balanced split -> preprocessing.py fits transformations on training data -> models.py trains the chosen algorithm -> evaluation.py computes test metrics -> visualization.py draws the decision boundary -> gui.py presents results. The report generator reuses the same project components to compare the full set of feature/class combinations.

Design benefit

Separating UI from the model and evaluation logic makes the project easier to test, debug, document, and extend without placing the learning algorithms inside GUI callbacks.

5. Graphical User Interface and User Workflow

The application uses Tkinter and has three main tabs: Train & Evaluate, Classify One Sample, and Full Analysis Report.

5.1 Train & Evaluate

6. Choose the Penguins CSV file (the supplied data/penguins.csv is the default project dataset).
7. Choose two different input features from CulmenLength, CulmenDepth, FlipperLength, OriginLocation, and BodyMass.
8. Choose two different species. The first is shown as Class A (-1); the second is Class B (+1).
9. Select Perceptron or Adaline.
10. Enter a positive learning rate and a maximum epoch count. Set the MSE threshold for ADALINE and check or clear Use bias.
11. Click Train & Evaluate. The interface displays the test accuracy, confusion matrix, model details, and decision-boundary plot.

5.2 Classify One Sample

After training, open the Classify One Sample tab. Enter the values of the same two features used by the trained model and request classification. The application displays the predicted species and a linear decision score. The score is not a probability estimate.

5.3 Full Analysis Report

The report workflow evaluates the full feature-pair/class-pair experiment set for both algorithms, chooses five representative plots for each algorithm, and creates analysis_report.md, analysis_report.html, experiment_results.csv, and ten PNG figures in output/.

Running the application on Windows

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python main.py
```

To generate the report directly from PowerShell, run python generate_report.py. To run the included tests, run python -m unittest discover -s tests -v.

6. Experimental Protocol and Evaluation

6.1 Experiment space

Five features produce 10 unique two-feature combinations (5 choose 2). Three possible pairs exist among the three species: Adelie vs Chinstrap, Adelie vs Gentoo, and Chinstrap vs Gentoo. The report generator evaluates the 10 feature pairs against all 3 class pairs for each algorithm: 30 experiments per algorithm and 60 total experiment rows.

Parameter	Automatic report setting	GUI behavior
Learning rate (eta)	0.01	User-editable; must be greater than zero.
Maximum epochs	150	User-editable; must be at least one.
MSE threshold	0.01	User-editable; used for ADALINE stopping.
Bias	Enabled	Checkbox allows enabling/disabling bias.
Random seed	42	Fixed seed supports repeatability for the same code/data.
Train/test samples	30/class and 20/class	The task-prescribed 30/20 split is enforced.

6.2 Confusion matrix layout

Rows represent the actual class and columns represent the predicted class. Class A corresponds to -1 and Class B corresponds to +1.

Actual \ Predicted	Predicted A (-1)	Predicted B (+1)
Actual A (-1)	True Class A	Class A predicted as B
Actual B (+1)	Class B predicted as A	True Class B

Metric definitions

Overall accuracy = (correct predictions) / (all test samples). For the required test split, the denominator is 40. Each matrix must sum to 40, and the diagonal gives the correct predictions. The code computes the matrix and accuracy manually; no scikit-learn metrics utility is used.

6.3 Reading a decision-boundary plot

- The straight line is the learned decision boundary that separates the model's two predicted regions.
- Filled circles represent training samples; hollow triangles represent held-out test samples.
- The figure title identifies the algorithm, the two classes, the selected features, and test accuracy.
- Well-separated test markers generally indicate a strong linear separation for that particular split; markers on the opposite side of the boundary indicate misclassifications.

The plot uses the same training-fitted imputation and standardization parameters as the classifier. As a result, it visualizes the decision surface of the trained model in the original feature coordinate space.

7. Experimental Results: Perceptron

Five representative examples are included below, ranging from the highest-performing examples to the weakest example in the report. The full CSV contains 30 Perceptron experiments across all feature/class combinations.

#	Classes	Features	Accuracy	Correct	Epochs	Final MSE
1	Chinstrap vs Gentoo	OriginLocation + BodyMass	100.0%	40/40	2	0.9672
2	Chinstrap vs Gentoo	CulmenLength + OriginLocation	100.0%	40/40	2	0.8727
3	Adelie vs Gentoo	CulmenLength + CulmenDepth	97.5%	39/40	2	0.9416
4	Adelie vs Gentoo	CulmenLength + BodyMass	90.0%	36/40	150	0.8513
5	Adelie vs Chinstrap	CulmenDepth + BodyMass	52.5%	21/40	150	0.9863

Interpretation

The displayed Perceptron examples span 52.5% to 100% test accuracy. Two selected Chinstrap/Gentoo combinations reach 100%, while CulmenDepth + BodyMass for Adelie vs Chinstrap reaches only 52.5%. A lower score does not by itself prove a code defect; it can reflect overlap between the chosen feature distributions and the limitation of a straight decision boundary.

Best observed accuracy

Highest observed Perceptron test accuracy: 100.0% (40/40 correct) in the automatic report. Multiple feature/class combinations tied at this score; see the final report CSV for the full set.

7.1 Perceptron Decision-Boundary Plots

1. Highest-performing example

Algorithm: Perceptron | Classes: Chinstrap vs Gentoo | Features: OriginLocation + BodyMass

Test accuracy: 100.0% (40/40 correct) | Epochs: 2 | Final training MSE: 0.9672

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[20, 0], [0, 20]]

Interpretation: All 40 test samples are correct. The selected classes are cleanly separated by the learned linear boundary in this fixed test split.

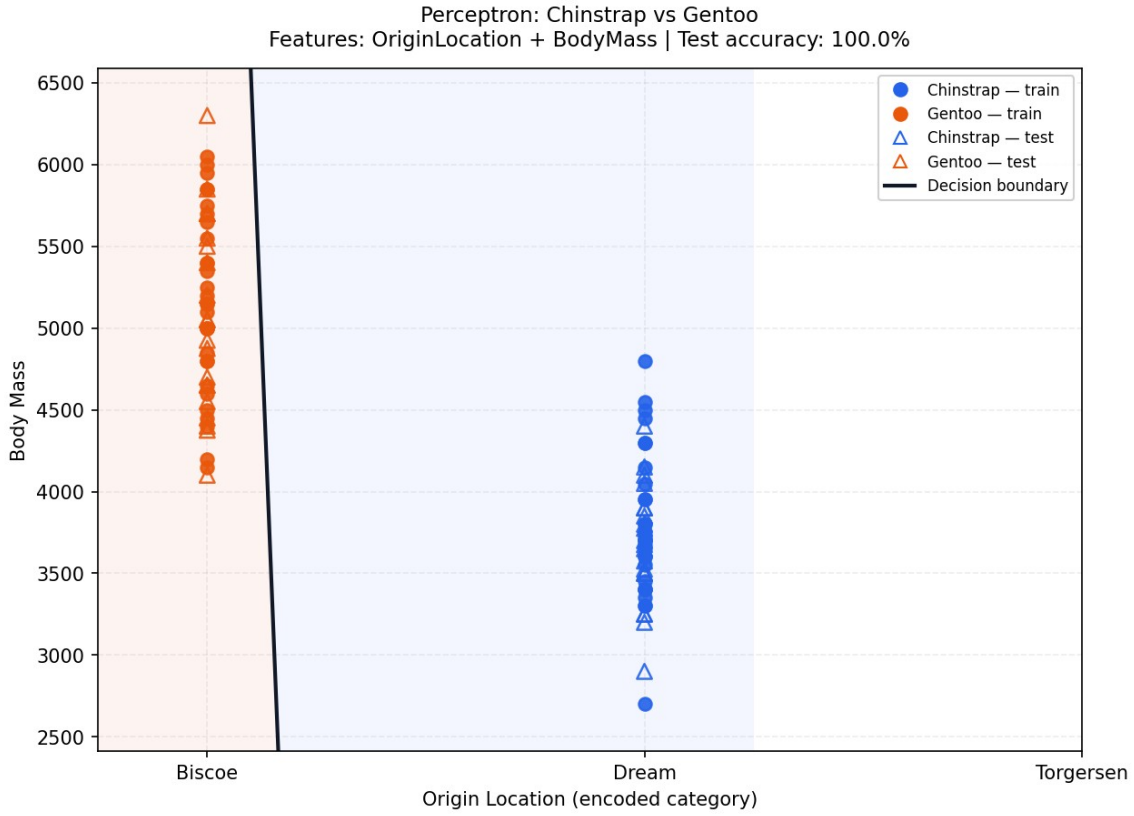


Figure 1. Perceptron: Chinstrap vs Gentoo; features OriginLocation + BodyMass; test accuracy 100.0%.

7.1 Perceptron Decision-Boundary Plots (continued)

2. Upper-range example

Algorithm: Perceptron | Classes: Chinstrap vs Gentoo | Features: CulmenLength + OriginLocation

Test accuracy: 100.0% (40/40 correct) | Epochs: 2 | Final training MSE: 0.8727

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[20, 0], [0, 20]]

Interpretation: All test samples are correct. The feature pair strongly separates the selected species on this split.

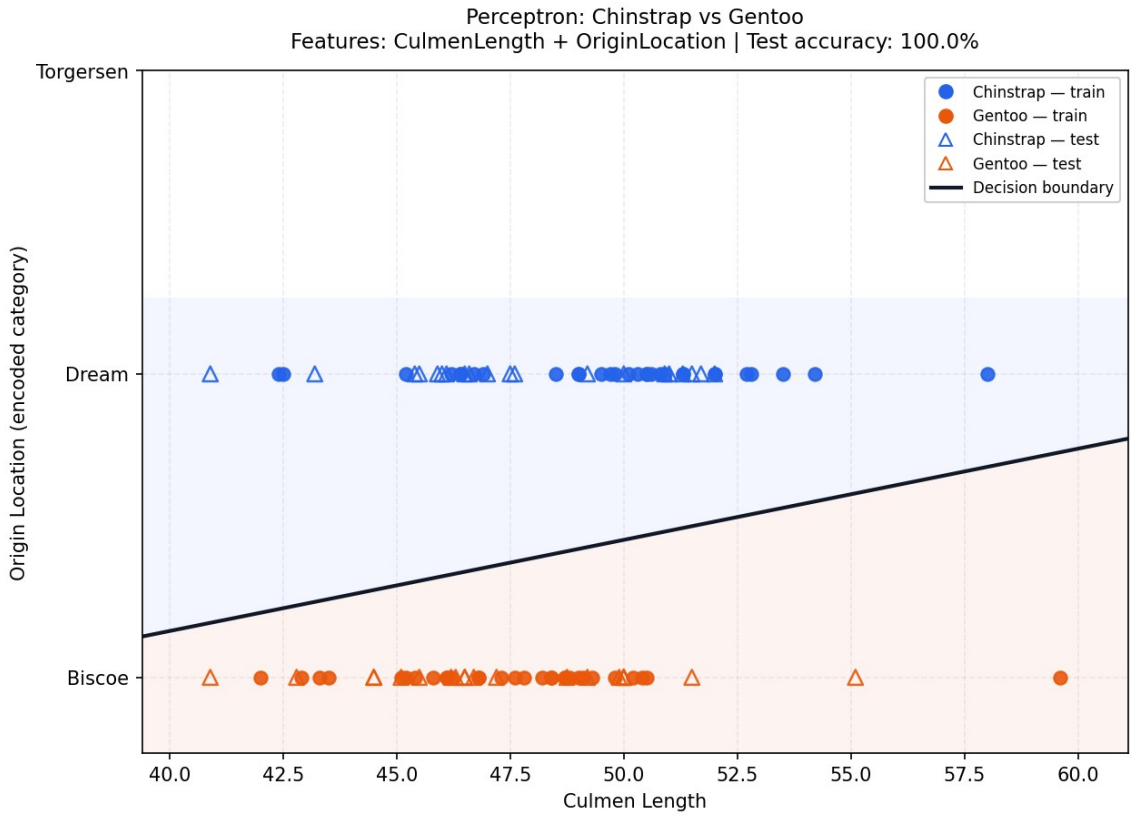


Figure 2. Perceptron: Chinstrap vs Gentoo; features CulmenLength + OriginLocation; test accuracy 100.0%.

7.1 Perceptron Decision-Boundary Plots (continued)

3. Middle-range example

Algorithm: Perceptron | Classes: Adelie vs Gentoo | Features: CulmenLength + CulmenDepth

Test accuracy: 97.5% (39/40 correct) | Epochs: 2 | Final training MSE: 0.9416

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[20, 0], [1, 19]]

Interpretation: One test sample from Class B is predicted as Class A; the remaining 39 samples are correctly classified.

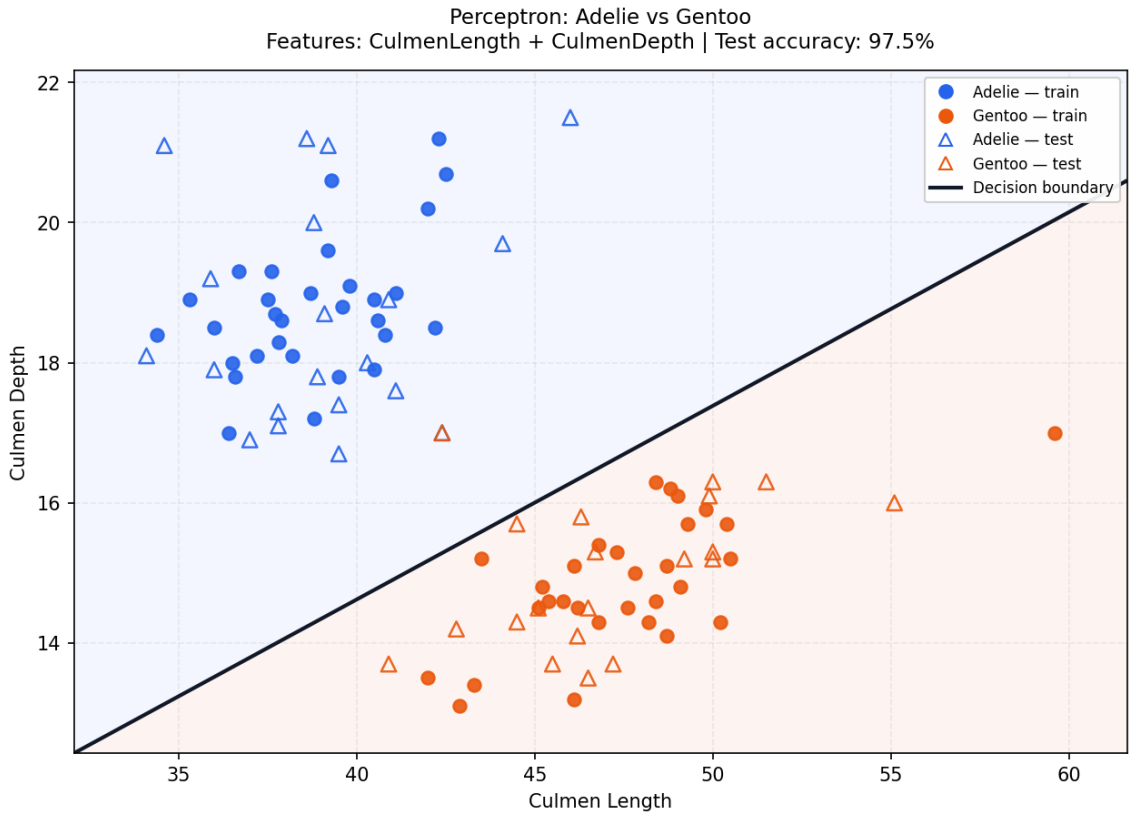


Figure 3. Perceptron: Adelie vs Gentoo; features CulmenLength + CulmenDepth; test accuracy 97.5%.

7.1 Perceptron Decision-Boundary Plots (continued)

4. Lower-range example

Algorithm: Perceptron | Classes: Adelie vs Gentoo | Features: CulmenLength + BodyMass

Test accuracy: 90.0% (36/40 correct) | Epochs: 150 | Final training MSE: 0.8513

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[18, 2], [2, 18]]

Interpretation: Four test samples are misclassified, with two errors in each direction. The combination is useful but not perfectly separable by the learned line.

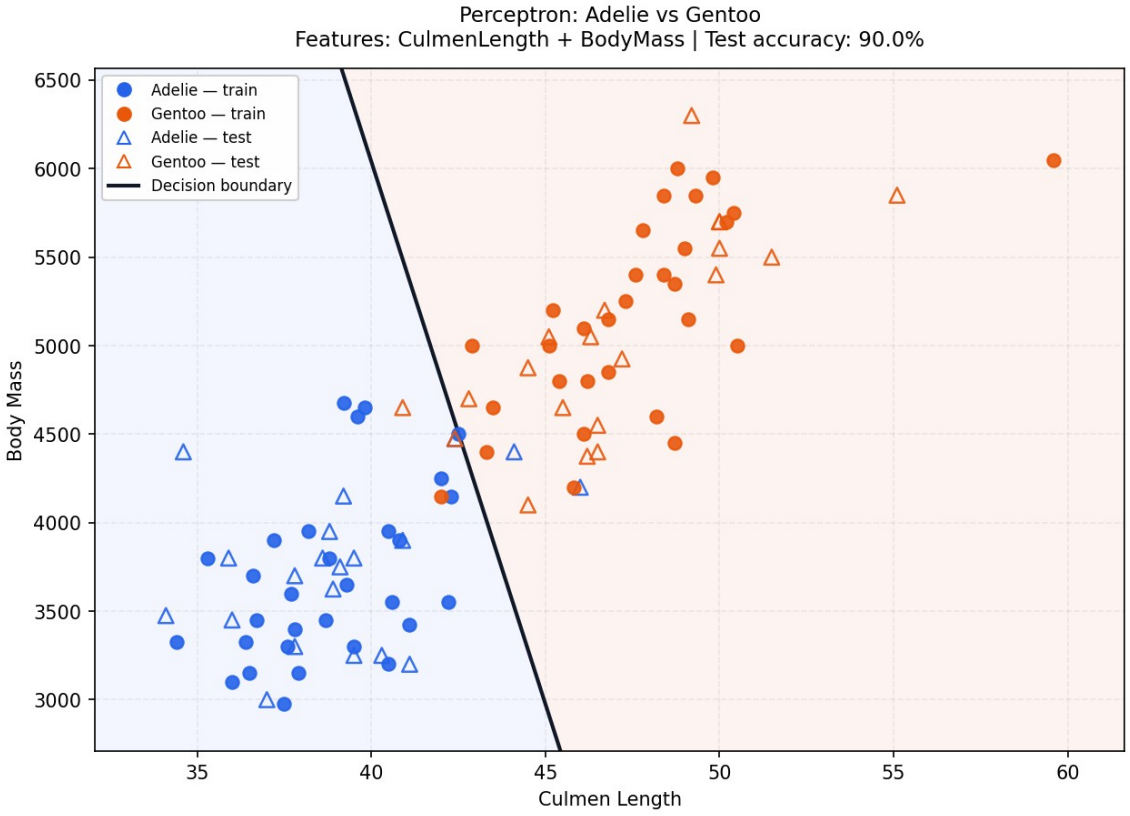


Figure 4. Perceptron: Adelie vs Gentoo; features CulmenLength + BodyMass; test accuracy 90.0%.

7.1 Perceptron Decision-Boundary Plots (continued)

5. Lowest-performing example

Algorithm: Perceptron | Classes: Adelie vs Chinstrap | Features: CulmenDepth + BodyMass

Test accuracy: 52.5% (21/40 correct) | Epochs: 150 | Final training MSE: 0.9863

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[8, 12], [7, 13]]

Interpretation: Only 21 of 40 samples are correct. The overlap and error counts indicate weak separation for this pair with this linear model and fixed settings.

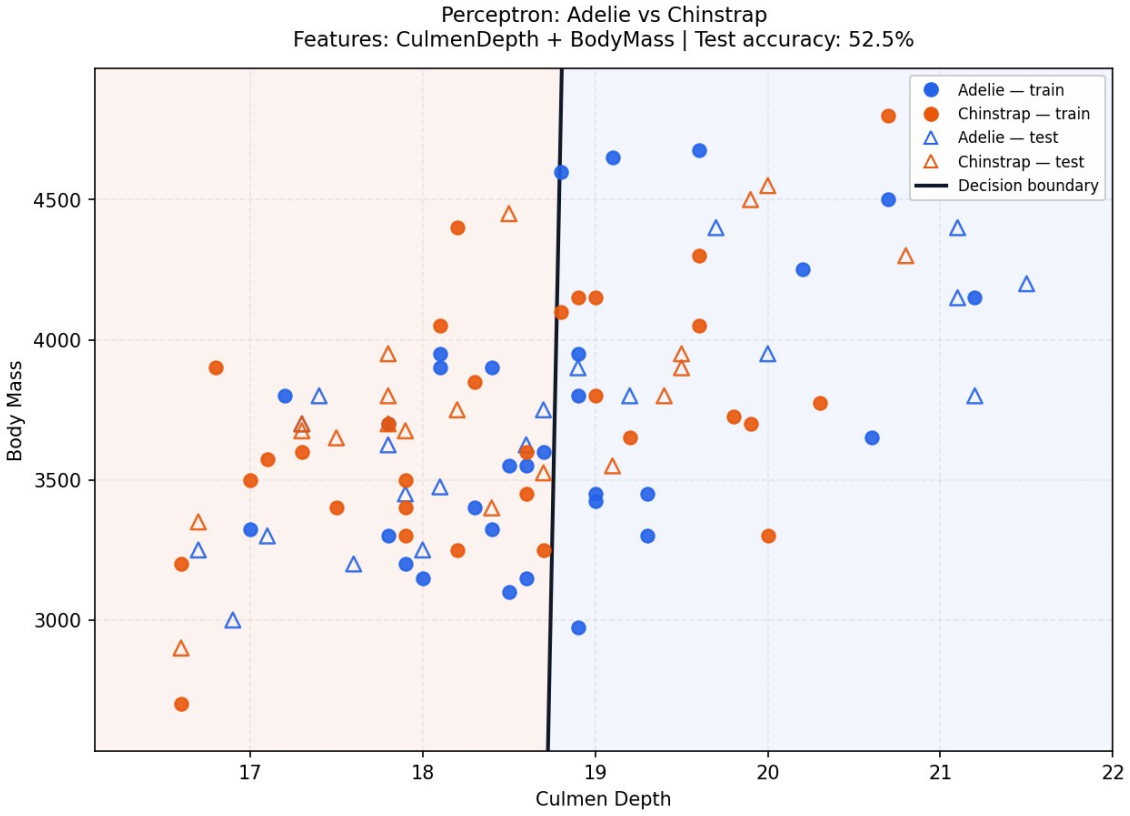


Figure 5. Perceptron: Adelie vs Chinstrap; features CulmenDepth + BodyMass; test accuracy 52.5%.

What this weak example shows

The confusion matrix records 8 correct Adelie predictions and 13 correct Chinstrap predictions; 19 of 40 samples are misclassified. The selected features do not yield a strong straight-line separation for this pair under the stated split and settings. In practical evaluation, accuracy must be interpreted together with the confusion matrix and the feature pair used.

8. Experimental Results: ADALINE

ADALINE learns from the continuous linear-output error and minimizes mean squared error by stochastic gradient descent. The five representative examples span high to low test accuracy. The complete CSV records 30 ADALINE experiments.

#	Classes	Features	Accuracy	Correct	Epochs	Final MSE
1	Chinstrap vs Gentoo	OriginLocation + BodyMass	100.0%	40/40	10	0.0095
2	Chinstrap vs Gentoo	CulmenLength + OriginLocation	100.0%	40/40	5	0.0051
3	Adelie vs Gentoo	CulmenDepth + FlipperLength	97.5%	39/40	150	0.0431
4	Adelie vs Chinstrap	CulmenLength + FlipperLength	90.0%	36/40	150	0.2122
5	Adelie vs Chinstrap	CulmenDepth + OriginLocation	50.0%	20/40	150	0.9716

Interpretation

The shown ADALINE experiments range from 50% to 100% test accuracy. The best displayed Chinstrap/Gentoo combinations both classify all 40 test samples correctly and have final training MSE below the 0.01 threshold. The weakest displayed combination (CulmenDepth + OriginLocation for Adelie vs Chinstrap) reaches 50%, showing that these features do not separate that class pair well in this run.

Best observed accuracy

Highest observed ADALINE test accuracy: 100.0% (40/40 correct) in the automatic report. Several feature/class combinations tie at the maximum; consult experiment_results.csv for all ties.

8.1 ADALINE Decision-Boundary Plots

1. Highest-performing example

Algorithm: ADALINE | Classes: Chinstrap vs Gentoo | Features: OriginLocation + BodyMass

Test accuracy: 100.0% (40/40 correct) | Epochs: 10 | Final training MSE: 0.0095

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[20, 0], [0, 20]]

Interpretation: All 40 test samples are correctly classified. The final MSE is below the configured 0.01 threshold.

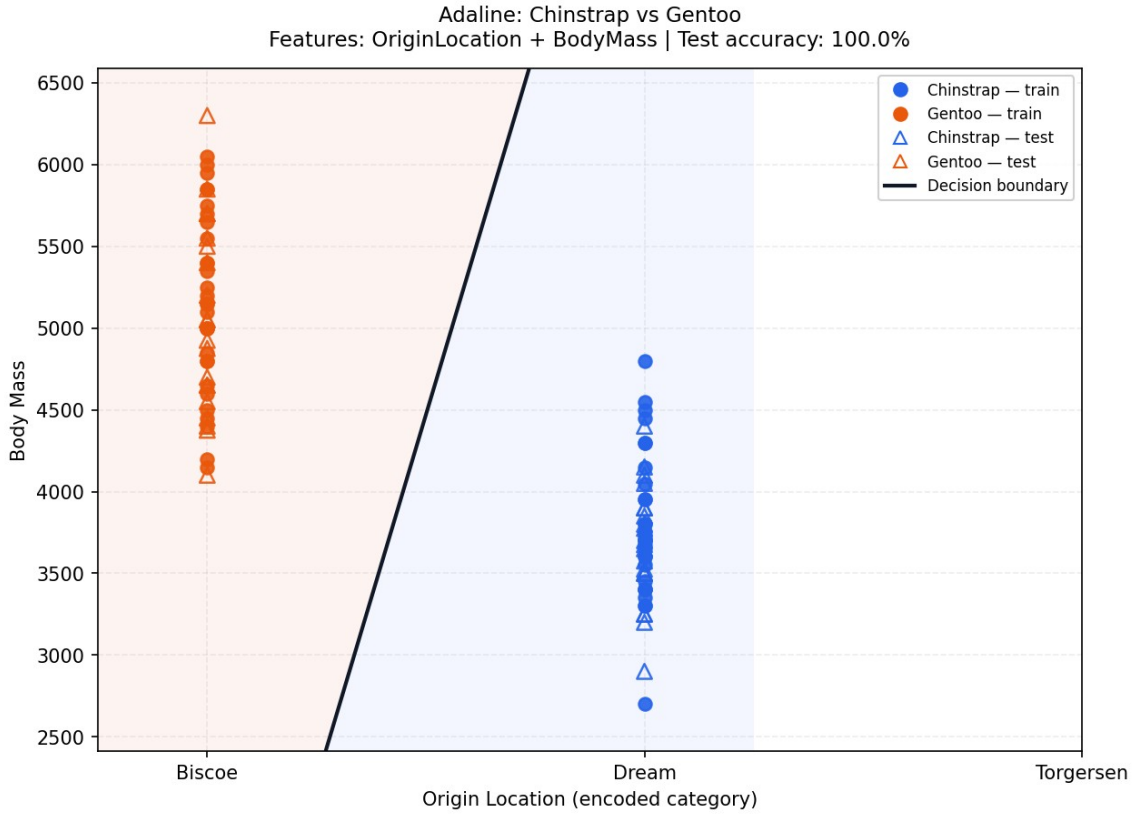


Figure 6. ADALINE: Chinstrap vs Gentoo; features OriginLocation + BodyMass; test accuracy 100.0%.

8.1 ADALINE Decision-Boundary Plots (continued)

2. Upper-range example

Algorithm: ADALINE | Classes: Chinstrap vs Gentoo | Features: CulmenLength + OriginLocation

Test accuracy: 100.0% (40/40 correct) | Epochs: 5 | Final training MSE: 0.0051

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[20, 0], [0, 20]]

Interpretation: All test samples are correct, and the training MSE reaches the threshold in five epochs.

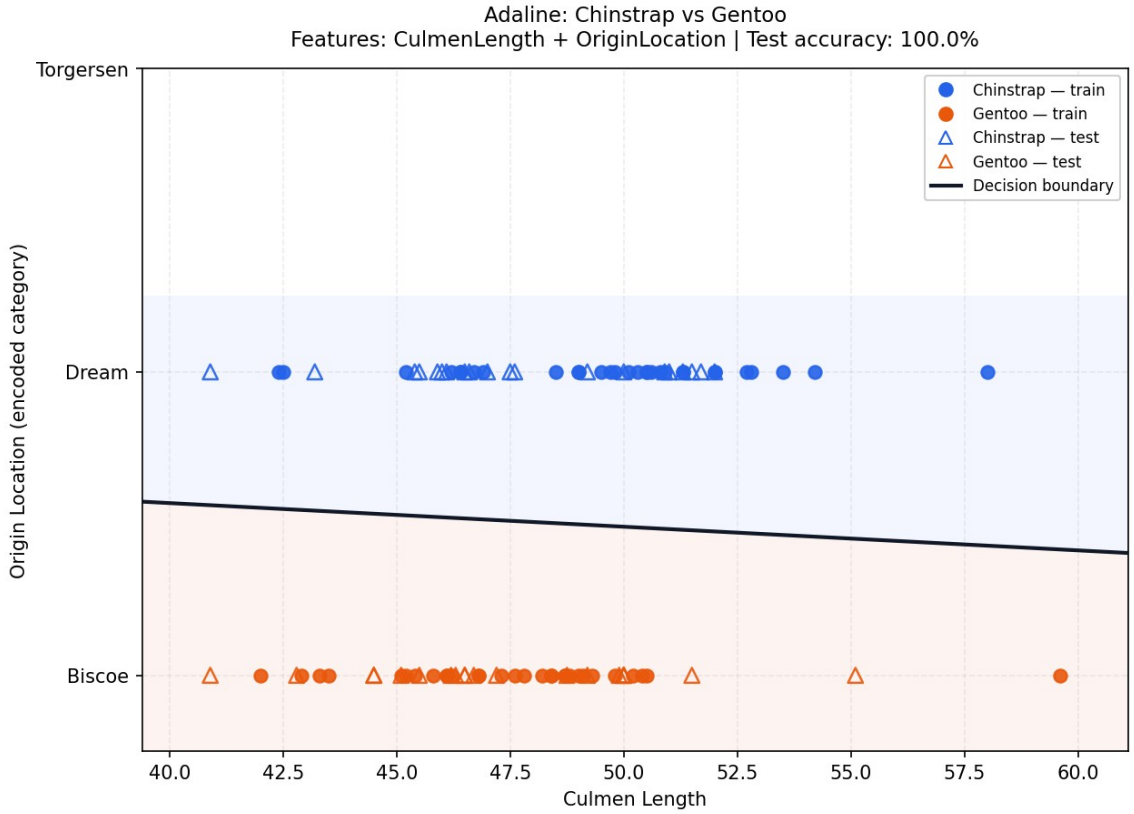


Figure 7. ADALINE: Chinstrap vs Gentoo; features CulmenLength + OriginLocation; test accuracy 100.0%.

8.1 ADALINE Decision-Boundary Plots (continued)

3. Middle-range example

Algorithm: ADALINE | Classes: Adelie vs Gentoo | Features: CulmenDepth + FlipperLength

Test accuracy: 97.5% (39/40 correct) | Epochs: 150 | Final training MSE: 0.0431

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[19, 1], [0, 20]]

Interpretation: One test sample from Class A is predicted as Class B. Test accuracy remains high even though the MSE threshold is not reached within 150 epochs.

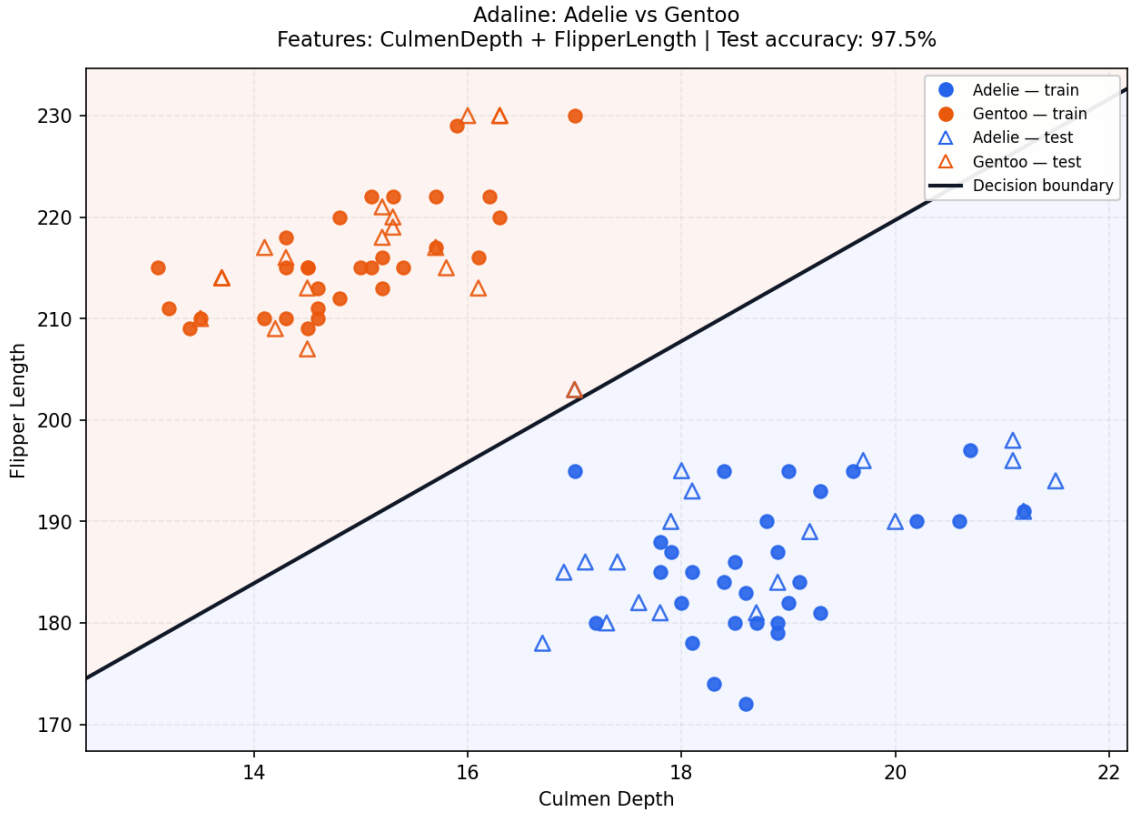


Figure 8. ADALINE: Adelie vs Gentoo; features CulmenDepth + FlipperLength; test accuracy 97.5%.

8.1 ADALINE Decision-Boundary Plots (continued)

4. Lower-range example

Algorithm: ADALINE | Classes: Adelie vs Chinstrap | Features: CulmenLength + FlipperLength

Test accuracy: 90.0% (36/40 correct) | Epochs: 150 | Final training MSE: 0.2122

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[18, 2], [2, 18]]

Interpretation: Four test samples are misclassified. The selected pair gives a useful but imperfect linear separation.

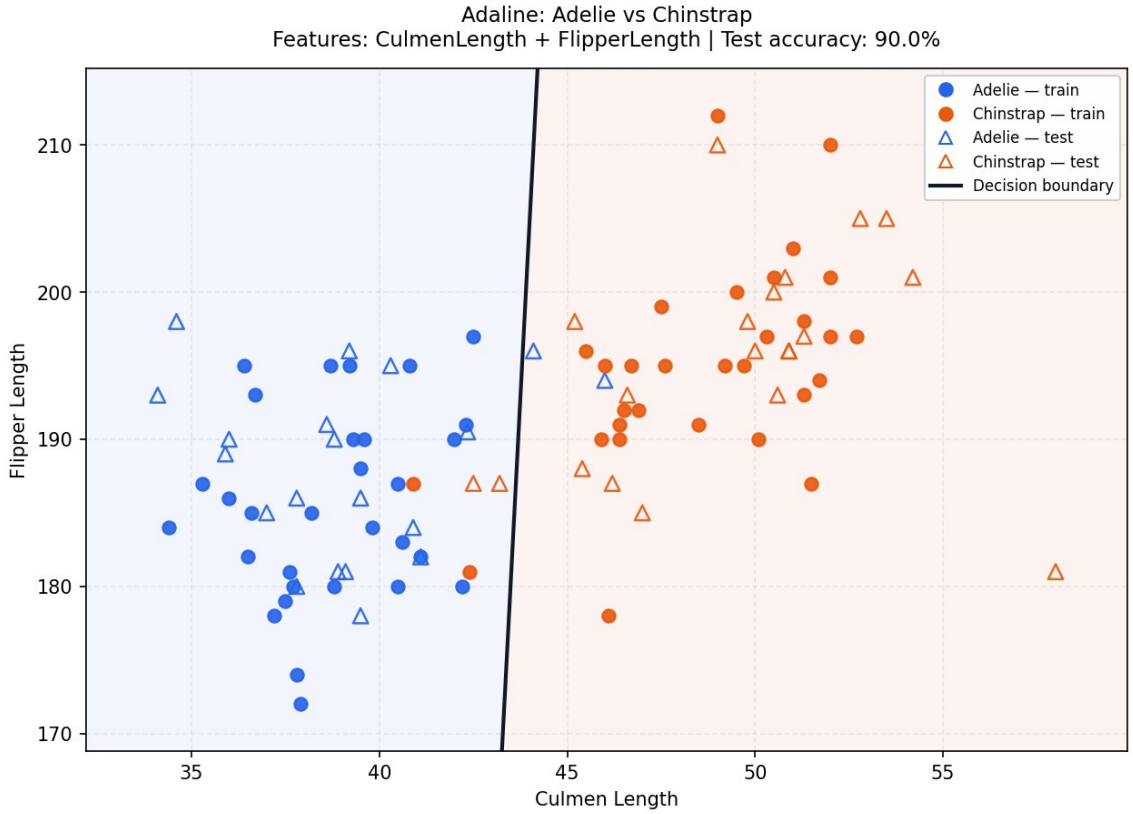


Figure 9. ADALINE: Adelie vs Chinstrap; features CulmenLength + FlipperLength; test accuracy 90.0%.

8.1 ADALINE Decision-Boundary Plots (continued)

5. Lowest-performing example

Algorithm: ADALINE | Classes: Adelie vs Chinstrap | Features: CulmenDepth + OriginLocation

Test accuracy: 50.0% (20/40 correct) | Epochs: 150 | Final training MSE: 0.9716

Confusion matrix (rows = actual Class A/B; columns = predicted Class A/B): [[8, 12], [8, 12]]

Interpretation: Only half the test set is correct, and both classes have substantial confusion. This is a weak example for the selected two-feature linear classifier.

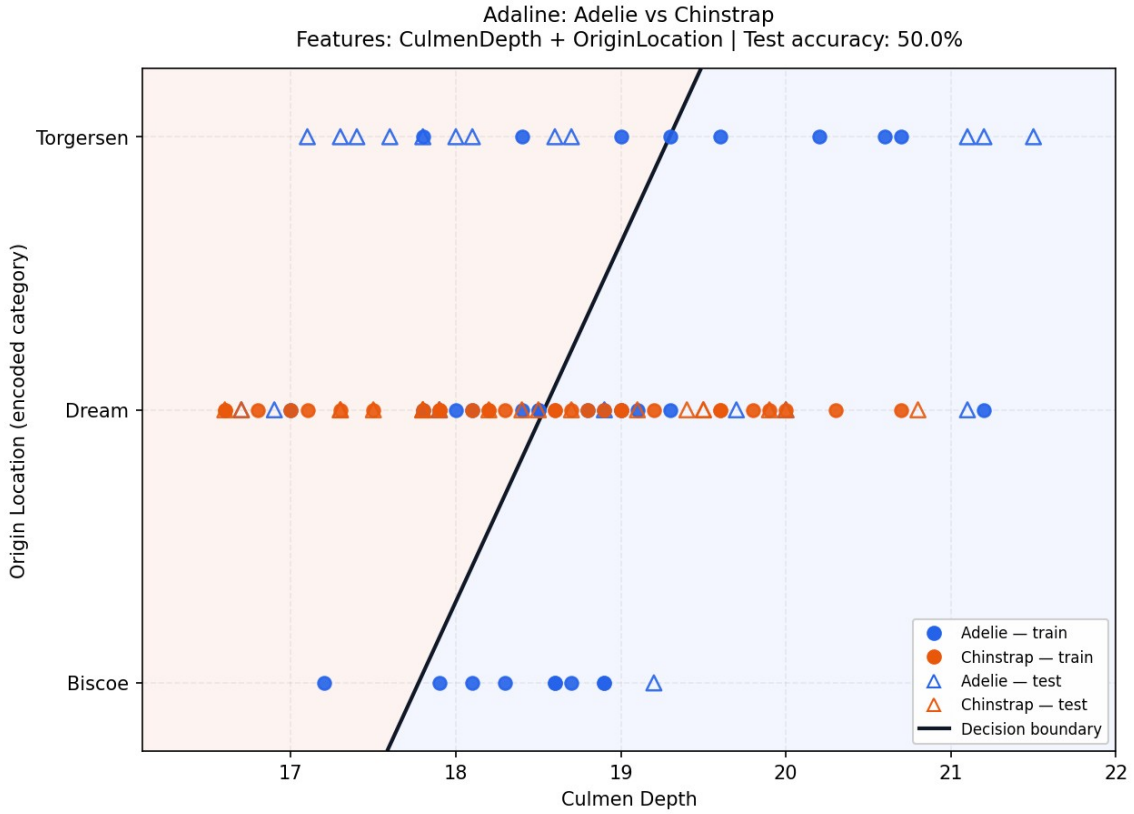


Figure 10. ADALINE: Adelie vs Chinstrap; features CulmenDepth + OriginLocation; test accuracy 50.0%.

What this weak example shows

The confusion matrix is [[8, 12], [8, 12]], so the model correctly classifies 8 samples from Class A and 12 from Class B, for 20/40 overall. The high number of errors in both directions indicates weak separation for this feature/class choice. The MSE threshold was not reached within the maximum 150 epochs.

9. Comparison, Testing, and Conclusions

9.1 Comparison of Perceptron and ADALINE

Comparison item	Perceptron	ADALINE
Highest observed test accuracy	100.0%	100.0%
Lowest displayed representative example	52.5%	50.0%
Primary learning signal	Discrete classification error.	Continuous linear-output error and MSE.
MSE behavior	Recorded as a diagnostic value; not used as the objective function.	Explicit optimization/early stopping target.
Shared limitation	Linear boundary may not fit overlapping feature distributions.	Linear boundary may not fit overlapping feature distributions.

9.2 Best-performing features

In the report's fixed-seed results, both algorithms reach 100% test accuracy for multiple feature/class combinations. OriginLocation + BodyMass and CulmenLength + OriginLocation for Chinstrap vs Gentoo are among the displayed 100% examples. The automatic report also reports tied best combinations involving other features and class pairs; the exact set is listed in the generated analysis_report.md and the all-combinations CSV. Therefore, no single feature pair should be described as universally best across every class pair.

10. Verification and Submission Checklist

10.1 Unit-test verification

The bundled core test suite was run during project verification. Six tests passed:

- The loader preserves the full row count and encodes OriginLocation.
- The balanced split contains 60 training samples and 40 testing samples with no overlap.
- Both models complete a binary classification experiment.
- Confusion matrix and accuracy are calculated by the custom implementation.
- Single-sample prediction returns one selected class.
- Single-sample prediction rejects an incorrect number of feature values.

10.2 Limitations and interpretation

- Results describe one fixed split and the stated training settings; changing random seed, learning rate, epochs, or bias may change performance.
- The classifier is deliberately linear and uses only two features at a time, so it cannot represent every possible class boundary.
- OriginLocation's scalar encoding is nominal, not ordinal; this affects how combinations using that feature should be interpreted.
- The decision score displayed for one sample is not a probability.
- A generated report confirms that the report pipeline ran, but final submission screenshots of the live GUI should be captured by the team from its own machine if the instructor expects GUI screenshots.

10.3 Submission checklist

Deliverable	Status / evidence
Python source files	Included as separate modules in the project folder.
Dataset	data/penguins.csv included; all 150 rows are retained on load.
GUI	Feature/class selection, algorithm selection, parameters, bias, results and sample prediction.
Visualization	Decision-boundary and scatter plots for representative experiments.
Evaluation	Manual confusion matrix and overall accuracy without scikit-learn metrics.
Analysis report	Markdown and HTML report plus experiment_results.csv and ten PNG figures.
Documentation	This team-specific Lab 3 document.

Final conclusion

The project implements the required Perceptron and ADALINE workflows, includes user-configurable GUI controls, preserves the dataset rows, uses the required balanced split, manually computes the evaluation metrics, and generates ten representative visual analyses. Accuracy depends on the selected classes, features, and training settings.